

CAManagement

caconsole — Operator Manual

Running a certificate authority whose signing key never leaves its token
— and provisioning the cards that read what it protects

Generated 2026-10-01

© 2026

Hebel Consulting GmbH

Schweighofplatz 7

6010 Kriens (LU)

Switzerland

support@simplarchive.dev

Contents

1	Introduction	1
2	Installing it	2
3	Connecting to a token	4
4	Standing up a CA	5
5	Issuing a certificate	6
6	Provisioning a reader's card	7
7	The enrolment manifest	9
8	Enrolling a key that never leaves the token	10
9	Revocation, CRLs and OCSP	11
10	Administering a token	12
11	Inspecting an artefact	13
12	Distributing an identity to an Apple device	14
13	Traps worth knowing	15
14	Scope	16

1 Introduction

caconsole runs a certificate authority whose **signing key never leaves its token**. It speaks PKCS#11 directly, authors the X.509 structures itself, and ships as one command.

Two properties shape everything in this manual, and both are stated as decisions rather than claimed as qualities:

- **The key stays in the token.** The library never sees private key material. It hands to-be-signed bytes to the token and takes back a signature — so there is no file to protect, no copy to lose, and no step at which a key could be exported by mistake.
- **We own the bytes.** Certificates, requests, CRLs and OCSP messages are authored field by field. The framework and OpenSSL stacks are used in the **tests**, as independent referees, and never at runtime.

1.1 What it is for

Two jobs, and they meet in the middle:

1. **Being a CA** — stand one up on a token, issue from requests, revoke, publish a CRL, answer OCSP.
2. **Provisioning the cards that read encrypted content** — generate a key on a smartcard, have the card prove it holds that key, issue it a **decryption** certificate, and record the issuance so an archive can enrol it in bulk.

The second is why this toolkit exists alongside SimplArchive: a reader's private key belongs on their own card, and somebody has to put it there and certify it.

1.2 Where the pieces live

package	
...CAManagement.Pkcs11	PKCS#11 v2.40 interop — sessions, key generation, sign and verify, encrypt and decrypt (including RSA-OAEP, for envelope-encryption unwrap), objects
...CAManagement.X509	X.509 and DER authoring: certificates, requests, CRLs, OCSP, chain validation, and an analyzer. No PKCS#11 dependency.
...CAManagement.Pkcs11.Signing	the adapter that lets the X.509 layer sign with a token key
caconsole	this tool

The two layers meet at one small seam, ICertificateSigner. Give the X.509 library an HSM-backed signer and it issues from a token; give it a software key and it issues from memory. **Nothing above the seam knows the difference**, which is why the library half is useful on its own.

2 Installing it

2.1 Two distribution forms, and which to take

you have	take
.NET installed — a developer machine, a build agent, a container on a .NET base image	the tool package : 1.2 MB, one package for every platform
no .NET at all — an administrator's workstation, a minimal host	the self-contained binary from scripts/publish.sh: 72–79 MB per platform, nothing to install

```
dotnet tool install --global HebelConsulting.CAManagement.Cli
```

command not found: caconsole does not mean the install failed, and it is the first thing most people hit. A global tool lands in ~/.dotnet/tools, which is not on PATH by default:

```
export PATH="$PATH:$HOME/.dotnet/tools" # add it to your shell profile to keep it
```

dotnet tool install prints that warning **once, in passing, on a first install only** — so on a machine that has ever had a global tool it says nothing at all, and on a fresh one it scrolls past.

Tell the two cases apart with dotnet tool list --global: if the table lists the package and the command, the tool **is** installed and only the PATH is missing. ~/.dotnet/tools/caconsole runs it meanwhile.

On Windows, use the self-contained binary rather than the tool package. The tool targets net10.0 only, because a tool package cannot carry a platform-specific target framework — and Windows needs its own, because there CK_ULONG is 4 bytes (LLP64) against 8 on Unix (LP64), and the structures pack differently too. A library consumer on Windows must likewise target net10.0-windows: a plain net10.0 application there would pick the LP64 build and **corrupt every CK_ULONG**.

2.2 A PKCS#11 module

For development, SoftHSM2 — installed through the host’s own package manager, never a piped installer or an unpacked download:

```
brew install softhsm      # macOS
apt-get install softhsm2  # Debian/Ubuntu
```

caconsole finds the module at the platform’s default location. Point it anywhere else — a card vendor’s PKCS#11 library, a production HSM — with `--module`.

3 Connecting to a token

Every command that touches the HSM shares four options:

option		default
--module <PATH>	which PKCS#11 module to load	the platform's SoftHSM2
--token-label <LABEL>	which token to use	the first slot with a token
--slot <ID>	an explicit slot id	overrides --token-label
--pin <PIN>	the user PIN	required — there is no interactive prompt

The PIN is read from the command line, and a command line is not private. It is visible to other processes on the host and it lands in shell history. Supply it from a controlled environment — a variable the shell does not record, a secret the CI system injects — and rotate it if it may have leaked.

This manual therefore writes PINs as placeholders ("`$CA_PIN`", `<pin>`) throughout rather than printing the development defaults.

A label that resolves to MORE THAN ONE token is refused, not guessed between. That sounds pedantic and is the opposite: a consumer whose provisioning created a second token under the same label would open a session on keys that **merely looked right** — for envelope encryption, wrapping against one token and failing to unwrap against another, which is silently unreadable data with every call returning success.

Found in the wild: a **“does the token exist?”** test anchored immediately after the label never matched, because SoftHSM **pads labels with trailing spaces**, so every restart initialised another token and three ended up sharing one label. Pass an explicit `--slot` where duplicates are legitimate, and see Section 10 for the repair.

4 Standing up a CA

The key is generated **on the token**; only the self-signed certificate comes out.

```
caconsole init-ca --token-label ca --pin "$CA_PIN" \  
  --label root --key-type ec \  
  --subject "/C=CH/O=Hebel Consulting GmbH/CN=Hebel Consulting Root CA" \  
  --years 10 --out ca.crt
```

--key-type is ec (P-256, the default) or rsa (2048). --subject accepts the slash form or the comma form, and per-component string-type annotations — CN=[PrintableString]www.example.org.

Keep the subject string. Anything that later has to chain to this CA must carry an issuer field matching this subject **byte for byte**. A leaf whose issuer differs does not chain, and on a mobile device the failure is a silent “*not trusted*” rather than an explanation.

5 Issuing a certificate

5.1 From a request

```
caconsole issue --token-label ca --pin "$CA_PIN" \  
  --ca-label root --ca-cert ca.crt \  
  --csr server.csr --days 365 --out server.crt
```

The requester's **proof-of-possession** signature is verified before anything is signed — a request that does not carry it is refused. Only the Subject Alternative Name extension is honoured **from** the request; everything else the certificate asserts is this command's to decide.

5.2 The certificate's PURPOSE, not its bits

```
caconsole issue ... --profile key-management
```

--profile takes signing (the default, and byte-identical to the behaviour before profiles existed) or key-management. The caller says what the certificate is **for**, and the keyUsage bit is **derived from the key's algorithm**:

profile	key	what the certificate declares
signing	any	digitalSignature, no extended key usage
key-management	RSA	keyEncipherment — key transport — plus emailProtection
key-management	EC	keyAgreement — ECDH key agreement — plus emailProtection

Naming the bits was rejected, because naming the bits is how the defect this fixes was written. A certificate issued for a card's Key Management slot once carried digitalSignature — the one bit that does not apply to it. Stating the purpose makes the wrong bit **unwritable**; stating the bits leaves it available.

A key algorithm that can do neither is **refused**, never quietly downgraded to a signing certificate, because a downgrade arrives looking like a success.

Why a wrong keyUsage is so easy to ship. It still works. Measured on hardware: a digitalSignature-only certificate **decrypted a real CMS envelope**, because neither the token nor most CMS libraries check keyUsage. So it fails only against consumers that **do** check — and then as a wrong-looking decryption error naming nothing at all.

The inverse is worth knowing if you write such a consumer: a certificate that works may legitimately declare the “wrong” usage, so refusing on keyUsage grounds alone will refuse certificates that function.

6 Provisioning a reader's card

One command takes a card from blank to enrollable.

```
caconsole yubikey provision --holder anna@acme.test \  
  --token-label ca --pin "$CA_PIN" --ca-label root --ca-cert ca.crt \  
  --label "YubiKey 5C" --manifest enrolments.json
```

It generates an **ECCP256** key in PIV slot **9d**, has the **card** sign a certificate request, issues a certificate for that key under the key-management profile (Section 5.2), imports it back onto the card, and reads the slot back to confirm.

6.1 Why slot 9D

PIV slot 9d is **Key Management** — the decryption slot, and the only one that can open a CMS envelope. 9a is authentication and 9c is signing.

Choosing the wrong slot is the classic first failure, and it fails late: everything provisions cleanly and nothing can read anything, much later, somewhere else.

Note also that the option is `--card-slot`, not `--slot`. `--slot` already means **the CA token's PKCS#11 slot**, and one option name with two meanings is how somebody re-keys a card while meaning to pick a token.

6.2 It will not quietly destroy a key

Where the slot is occupied, the command **prints what is there and stops**. Two flags open it up, and they are deliberately not one:

flag	
<code>--force</code>	replaces the certificate and keeps the key . The repair for a card whose certificate declares the wrong usage.
<code>--regenerate-key</code>	replaces the key . Irreversible — anything encrypted to the old one becomes unreadable.

Re-issuing a PIV certificate is non-destructive. Only key GENERATION destroys a key.

That is worth stating on its own line, because the instinct on finding a wrong certificate is to start over — and starting over is the one action that cannot be undone. A wrong-usage certificate is repaired with `--force`, and the person keeps their key, their enrolment and everything already addressed to them.

Any certificate already in the slot is exported beside the new one before it is replaced. An identity artefact is one-shot: the copy on the card is usually the only copy, and re-issuing over it would destroy the record of what was previously valid — which is exactly what somebody needs when asking whether an envelope sealed last year was addressed to a key that was valid then.

6.3 The pre-5.3 firmware caveat

A PIV applet older than 5.3 reports Private key type: EMPTY **whether or not a key is present** — it simply does not publish key metadata. Measured on a 5.2.6 card whose key decrypts real envelopes, and which reports Public key type: RSA2048 in the same breath (read from the slot's **certificate**, so it is evidence the slot is occupied rather than proof the key is reachable).

The key IS there, and only a decryption settles it. caconsole therefore treats EMPTY on such a card as **unknown**, not **absent**, and **refuses to generate** without `--regenerate-key`. Three states, not two, and **unknown is treated as occupied**: refusing a genuinely empty slot wastes a flag, while generating over a key that was there destroys the only copy.

To settle it yourself, verify the request the card signed — the signature is proof of possession:

```
openssl req -in <request>.csr -noout -subject -verify
```

6.4 Two more things the hardware imposes

Leave the touch policy off. A touch policy on 9d means a physical tap per **decryption** — per document opened, and per preview. Reasonable for signing, unusable for reading an archive.

A card cannot be provisioned over PKCS#11 at all, which is why this command drives the card vendor's own CLI underneath. The vendor's PKCS#11 module implements no key generation for PIV slots, cannot see an **empty** slot (it enumerates by certificates), and does not support token initialisation. Provisioning is the PIV applet protocol.

So ykman is a **named, checked prerequisite**: install it through the host's package manager (`dnf install yubikey-manager`, `apt-get install yubikey-manager`, `brew install ykman`), and the refusal names the package rather than offering a download. The command **hides the mechanism**, so a later move to native PIV changes nothing for you.

6.5 What is logged, and what is removed from it

Every card command and its response are **echoed**, because an administrator is entitled to see what was done to their token. The PIN and management key ride on the command line, so they are removed **by option name** — `-P`, `--pin`, `-m`, `--management-key` and their siblings — in both the separated and the attached spellings. The flag survives and its value becomes `***`, so the log still shows **that** a credential was supplied and **which** option carried it.

Why by name, and not the two obvious alternatives. Redacting by **position** breaks the moment options are reordered, and an option order is nobody's contract. Redacting by **value** — find the PIN string and blank it — blanks the **slot** argument when the PIN happens to be 9d, which is a legal PIN and a real slot, so the command then reads as though it operated on nothing.

An option this list does not recognise is logged **in full**. That fails visibly: a new secret-bearing option shows up in a log where somebody can see it and add it here, rather than being hidden along with the leak.

7 The enrolment manifest

provision and revoke can append to a JSON file recording **what was issued and to whom** — which is what a document archive needs in order to enrol certificates in bulk:

```
{
  "issued": [
    { "holder": "anna@acme.test", "label": "YubiKey 5C", "serial": "2CEF6410C6745437",
      "thumbprint": "CA5F...", "certificatePem": "-----BEGIN CERTIFICATE-----..." }
  ],
  "revoked": [ { "serial": "2334CB43CDEC7428", "at": "2026-09-30T19:36:21Z", "reason":
    "KeyCompromise" } ]
}
```

SimplArchive consumes it with one command:

```
saconsole certificates import --manifest enrolments.json
```

Four properties, each deliberate:

- **Separate from `ca-state.json`**, which holds only what the CA needs to run — the CRL number and its revocation list — and carries no issued certificates and **no e-mail addresses at all**.
- **Rows are upserted by serial**, so re-provisioning a card replaces its row rather than adding a second and leaving a reader to work out which is current.
- **Only the public certificate is ever written**. A test asserts that no PRIVATE KEY reaches the file even when the certificate object in hand holds one — because this is a file an operator copies to another machine and posts over a wire.
- **A serial keeps its leading zeros**. A serial is an opaque octet string rendered as hex, not a number, and a real card's begins with four zero bytes. Trimming them is the reflex, and it would make that certificate fail to match its own enrolment.

The shape is not shared through a package, on purpose. Each side defines it, and each side tests against a **literal fixture** of the file. A test reading both sides from one source agrees at every version while the wire disagrees — so a drift here shows up as one of the two fixtures failing, rather than as an import that silently enrolls nothing.

8 Enrolling a key that never leaves the token

Where the key is already on a token — a card provisioned elsewhere, a token you administer — the whole loop runs over PKCS#11 alone, so it works with any card the module can drive:

```
caconsole csr --token-label card --pin "$CARD_PIN" --label holder \  
  --subject "C=CH, O=Example, CN=Card Holder" --out holder.csr  
caconsole issue --token-label ca --pin "$CA_PIN" --ca-label root \  
  --ca-cert ca.crt --csr holder.csr --out holder.crt  
caconsole import-cert --token-label card --pin "$CARD_PIN" --label holder --cert holder.crt
```

The private key is never read at any step. `csr` signs **on** the token, which is also the request's proof of possession — and the decoder verifies that signature and refuses a request that does not carry it.

Why this exists. `issue` could always sign a request; until `csr` there was no way to **produce** one from a token-resident key, so enrolling a smartcard meant the card vendor's own tool — which is per-vendor, and therefore per-customer.

What is still the card tooling's job is **key generation and card personalisation** — the PIV card objects, the slot choice. Those are card-applet operations rather than PKCS#11 ones, and `caconsole` deliberately does not reach for them. Section 6 is the command that does.

9 Revocation, CRLs and OCSP

```
caconsole revoke --serial 0A1B2C --reason KeyCompromise --manifest enrolments.json
caconsole gen-crl --token-label ca --pin "$CA_PIN" \
  --ca-label root --ca-cert ca.crt --days 7 --out ca.crl
```

Reasons: Unspecified, KeyCompromise, CaCompromise, AffiliationChanged, Superseded, CessationOfOperation, CertificateHold, RemoveFromCrl, PrivilegeWithdrawn, AaCompromise.

9.1 Answering OCSP

One-shot, against a DER request:

```
caconsole ocsf-respond --token-label ca --pin "$CA_PIN" \
  --ca-label root --ca-cert ca.crt --reqin request.der --respout response.der
```

...or as an HTTP responder, serving both the POST and GET forms:

```
caconsole ocsf-respond --token-label ca --pin "$CA_PIN" \
  --ca-label root --ca-cert ca.crt --listen http://127.0.0.1:8080/ --validity-hours 24
```

A serial recorded by revoke reports revoked. **Any other serial of this CA reports good** — issuance is not tracked, which is worth knowing before that answer is relied on. A foreign issuer reports unknown.

9.2 Where CA state lives

revoke, gen-crl and ocsf-respond share the CRL number and the revocation list through --state:

--state	
ca-state.json	the default — a JSON file next to you
token	a data object on the HSM, so the bookkeeping travels with the key and is reachable only after login. Needs --ca-label and --pin as well.

CRL numbers need a persisted counter for strict monotonicity, which the CLI has — so a CLI user is safe and can stop reading here.

A **library** caller should know the rest. CrlBuilder's default CRL number is seconds since the epoch, which is a convenience for callers that supply none; it is monotonic only while you issue **at most one CRL per second** and **the clock never moves backward**. A time step-back or a snapshot restore could produce a lower number, which relying parties may treat as stale. Beyond one CRL per second, pass an explicit number.

10 Administering a token

Four commands cover the day-to-day jobs, through standard PKCS#11 calls, against **any** module:

```
caconsole list-slots
caconsole init-token --free --label ca --so-pin "$SO_PIN" --pin "$PIN"
caconsole set-pin    --token-label ca --pin "$PIN" --new-pin "$NEW_PIN"
caconsole wipe-token --slot 3 --so-pin "$SO_PIN" --new-label retired-2026-09-23 --yes
```

--free picks the first uninitialised slot; --slot <id> names one.

Prefer these to the module's own utility when a service is involved. caconsole resolves slots through the same code a consuming service uses, so what it reports is what that service will do. softhsm2-util answers about whatever its **own** configuration points at — which is usually not the service's token store, and which then **looks like an empty, healthy machine**.

10.1 wipe-token, and the case it exists for

Token **deletion** is a vendor extension rather than a standard PKCS#11 function, so it is out of scope — but the job it is usually wanted for is not. wipe-token re-initialises a token **in place**: every object on it is destroyed and it takes a new label.

That settles the case that actually bites — **several tokens sharing one label**, where a PKCS#11 caller resolves the name to whichever slot enumerates first — because the wiped ones stop answering to it. What it cannot do is remove the **slot**; nothing in PKCS#11 can, which is exactly why the verb is not called delete-token.

It is addressed by --slot only, never by label. The label is the thing in dispute, so resolving by it would pick between the duplicates at random.

It needs the token's existing SO PIN, refuses to run without --yes, and leaves behind a token with **no user PIN** — run init-token against it, or park it under its new label.

Wipe to DISTINCT labels. Retiring three ambiguous tokens to one shared retirement label simply recreates the ambiguity under a new name. And there is no undo: read the slot list first, and be certain which slot you are naming — if you cannot tell which token holds the keys your data references, **stop**.

11 Inspecting an artefact

asn reads PEM or DER and renders the structure on the left with a plain-English explanation on the right.

```
caconsole asn server.crt
caconsole asn /etc/ssl/cert.pem --index 42    # pick one certificate out of a bundle
caconsole asn ~/.ssh/id_ed25519              # decoded, not just rejected
```

It recognises certificates, PKCS#10 requests, CRLs, SubjectPublicKeyInfo, PKCS#8 / PKCS#1 / SEC1 private keys, PKCS#12 containers, CMS and PKCS#7, OCSP requests and responses, and OpenSSH keys.

asn decodes; it does not decrypt. A password-protected container — a PKCS#12 bag, an OpenSSH key with a passphrase — is shown as structure plus its key-derivation parameters. The secrets stay opaque.

This is the command to reach for when an interoperability argument needs settling: fetching an artefact and reading its actual bytes is diagnosis of a **format**, and it answers questions no status code can.

12 Distributing an identity to an Apple device

`caconsole mobileconfig ...`

Builds an Apple configuration profile from a PKCS#12 identity and/or root certificates — the one-tap install path for Apple devices. No token is involved.

A profile normally carries two payloads, and the second is easy to forget: the **identity**, and the **CA trust**. Without the trust payload the identity installs and nothing chains — and on a device that additionally reaches a server over TLS signed by the same CA, the trust payload is what makes **that** acceptable too. A missing one shows up as an unexplained “*not trusted*”.

13 Traps worth knowing

Each of these cost real debugging, and each is in the shape that makes it expensive: the symptom points somewhere else.

A mechanism/key-type mismatch is a PROCESS CRASH, not an error code. Asking SoftHSM to sign with an RSA mechanism against an EC key **segfaults inside the module** — exit 139, zero output, only the operating system’s crash report to read.

So the signer validates the key type against the requested algorithm **at construction** and throws a message naming both sides; and `Pkcs11CertificateSigner.ForKey` derives the algorithm **from the key**, so a “sign with this key” caller cannot mismatch at all. Prefer `ForKey` in library code.

SOFTSM2_CONF must be set in the real process environment. On macOS, the framework’s `SetEnvironmentVariable` does **not** reach native `getenv`, so a module ignores a managed-only setting and silently falls back to its default configuration — which is a different, usually empty, token directory. Set it before launch.

CKM_ECDSA_SHA256 is not in every SoftHSM build — present in some, absent in others, across platforms. An EC CA works against a token that implements the mechanism; where it is missing, use an RSA CA. This is a property of the build you installed, not of the key you chose.

Enumerating certificates needs no login; enumerating keys does. Certificates are public objects, so “*which certificates does this token carry?*” is answerable before a PIN is collected. `CKO_PRIVATE_KEY` objects are invisible until login — so “*does a key for this certificate live on this token?*” **cannot** be answered first, which matters when designing a prompt.

And enumerate by **class**, not by label. A card may carry a certificate in each of several slots under vendor-specific labels, and a consumer narrowing by label hard-codes one vendor’s naming and sees **nothing** on a token that names things differently — a silent empty result rather than an error. A person may also have **two cards plugged in**, so the unit to walk is the slot list, then the class query per slot.

14 Scope

Stated plainly, because a tool that pretends is worse than one that refuses.

- **PKCS#11 2.40** — the version SoftHSM2 implements. 3.0 is additive and can be layered on later without reworking what exists.
- **Not a policy engine.** The library issues exactly what you ask it to. Name constraints, path-length enforcement beyond what you set, and issuance policy are the caller's responsibility.
- **CertificatePolicies** supports the policy OID with an optional CPS URI and user-notice text; `noticeRef` is intentionally unsupported.
- **Issuance is not tracked.** The CA state records revocations, not issuances — which is why an unknown serial of this CA answers good to OSCP, and why the enrolment manifest exists separately for the archive's purposes.
- **Card personalisation beyond slot 9D** — the PIV card objects, other slots — is the card tooling's job, by decision rather than omission.

caconsole 1
Certificate authority 1

PKCS#11 1